

CISC484 Project 4 Report

Peter Chapman

University of Delaware

CISC484: Intro to Machine Learning

Project 4: Benthic Habitat Segmentation

December 9, 2025

Introduction

The territory of Guam contains several different types of habitats across its entire landmass, especially near the waters on the coastline. Like other coastal landmasses, several satellite images have been collected from Guam's coastlines. This project analyzes images from two specific regions near the coast: the Agana Bay and the Manell-Geus watershed. The Agana Bay contains shallow coastal waters (depth of 1-5 meters) with mixed seagrass beds and patchy coral formations. The Manell-Geus watershed contains a barrier reef system (depth of 5-25 meters) with depth-dependent habitat zonation. The goal of this project is to develop a deep learning-based model for benthic habitation segmentation and classification, using nine images from these two regions in Guam.

Methodology

Classes

For both of these two regions, there are seven habitats that serve as the target classes for the segmentation process. Table 1 below shows these seven different habitats, their associated colors on the labeled habitat maps, and the relative frequency of each habitat class across the entire labeled dataset. The frequencies indicate the distribution by pixels across all training masks. As one can see, there is a significant class imbalance between the seven habitats within the two regions. The first two habitats are the smallest by frequency. Class 0, the Background, contains sparse non-habitat areas and makes up 3.5% of the data. Class 1, the Seagrass, is a critical but rare habitat that forms just 3.06% of the dataset. The next three habitats are larger than the first two, but not quite as big as the last two. Class 2, the Coral, refers to ecologically vital communities that make up 7.65% of the data. Class 3, the Macroalgae, is a common reef component that forms 11.91% of the dataset. Class 4, the Sand, is the prevalent seafloor substrate that makes up 8.6% of the dataset. Finally, the last two habitats carry the largest frequencies of the seven habitats. Class 5, the Land, is essentially a dormant miscellaneous class that includes land and more. This habitat makes up 37.89% of the data. Class 6, the Ocean, includes the open water regions and forms 27.41% of the dataset. These seven habitats, their colors, and their frequencies were used for the deep learning models.

Number	Name	Color	Frequency (% of Data)
0	Background		3.5
1	Seagrass		3.06
2	Coral		7.65
3	Macroalgae		11.91
4	Sand		8.6
5	Land		37.89
6	Ocean		27.41

Table 1: The classification and distribution of the seven habitats that serve as target classes for the segmentation process.

Image Sizing

A problem arose from the nine images of the two regions in Guam: the raw training images are of different sizes. Some images are as small as 528x447 pixels, while some are as large as 1370x2051 pixels. This prevents the ability to pass the images into the same neural network model, so a consistent size must be developed across all the raw images. As discussed briefly in class, the first thought was to compress the images down to a certain size. In lecture, this was determined to be a nonoptimal way to solve this problem. Compressing or stretching the images results in a loss of quality and key data that is necessary for the ideal deep learning models to perform well.

The determined solution to this problem was to use patch subsampling for each image, which is when small pieces of the image are considered by the model one at a time. This allows for images of different sizes to be passed into the same model, as long as the pieces of each image are the same size when passed in. The image preparation step completed this process by going through each raw image and creating random patches from each image. This method of randomization allowed for many training samples per image, which is ideal considering the small number of complete images that were provided originally.

Spectral Bands

The second large problem with this project was the fact that each satellite image from the two regions in Guam contains eight distinct spectral bands, which each correspond to a different frequency of light. Unlike a typical RGB image that has three values (red, green, and blue), these multispectral images contain a much richer set of information. This creates many challenges for an image segmentation task such as this one. If unaddressed, a segmentation model could weigh certain bands too heavily or fail to learn specific spectral relationships, leading to poor class discrimination.

The determined solution for this problem was to calculate the average and the standard deviation for each of the eight bands in the images. With these averages and standard deviations in mind, each image was then normalized. Overall, this process should reduce the potential complexity introduced by having the eight spectral channels in the images.

Class Imbalance

The third and final problem is one that has already been identified above. Two habitats completely dominate the class distribution, while other ones are very rare. The Land and Ocean habitats together account for 65.3% of all pixels, while the ecologically critical habitats (Seagrass and Coral) represent less than 11% of the pixels combined. This is a challenge because

a highly accurate model could simply predict the Land and Ocean habitats a majority of the time, while not even considering the Seagrass or Coral habitats.

The solution to this problem was to determine the weight of each habitat using the provided class frequencies in Table 1 above. The inverse-frequency weights of the seven habitats are shown below in Table 2. By including these weights in the image preparation step, the two rare habitats will be ignored less frequently during the training process.

Number	Name	Color	Inverse-Frequency Weights
0	Background		28.57
1	Seagrass		32.68
2	Coral		13.07
3	Macroalgae		8.4
4	Sand		11.63
5	Land		2.64
6	Ocean		3.65

Table 2: The classification and inverse-frequency distribution of the seven habitats that serve as target classes for the segmentation process.

U-Net

The first convolutional neural network model considered was the U-Net model, which was recommended during lecture from the professor. This model, originally designed for biomedical image segmentation, was selected due to its ability to perform well with limited annotated data. The U-Net model essentially has two main components: an encoder block and a decoder block. Figure 1 below shows the image from class that served as inspiration for the U-Net model that was created. The goal was to create each of the aspects of the U-Net model. The encoder downsamples layers into small pieces. Next, the bottleneck between the encoder and the decoder is the deepest layer within the entire model. Finally, the decoder upsamples layers into larger images until the final image is produced as the output.

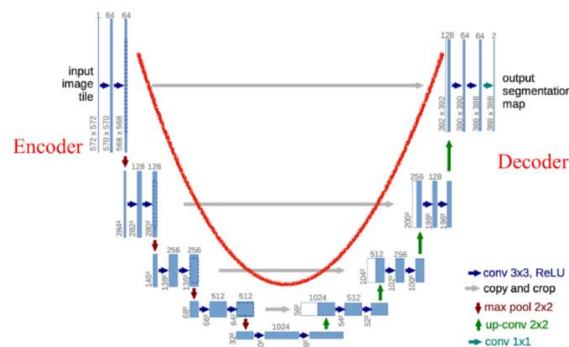


Figure 1: U-shaped encoder-decoder structure from lecture, inspiration for the created U-Net model for image segmentation.

Creating the code for the U-Net model was initially very tricky, but a YouTube video was found that demonstrated how to utilize PyTorch for an image segmentation task such as this one. The [video](#), created by Aladdin Persson on YouTube, details a U-Net convolutional neural network model architecture for an image segmentation task such as this one. A few adjustments had to be made to Persson's work, as his U-Net model addresses a different task from this one.

For training the U-Net model, the train/test ratio was seven images to two images in an attempt to emulate the traditional 80/20 split discussed in class. Two initial issues arose during this training process, which both adjusted the methodology used. First, there was an issue where only three of the nine images from Guam were considered in the training process. Second, the kernel kept crashing due to the lengthy run times that resulted from originally having 50 epochs used. To fix this, a clarification had to be made on the number of epochs, number of batches, patch size, and the number of patches to use for training. 10 epochs, 16 batches, a patch size of 256 pixels, and 15 patches per image were used. This allowed the kernel to function properly throughout the entire training of the model.

The equation below shows the inspiration for the cross-entropy loss function that was defined for use within the training script. This equation, as instructed in class, is a binary cross entropy loss function. For this image segmentation task, there are more than two outcomes due to the seven different habitats in the two regions of Guam. Therefore, a multi-class cross entropy function was created instead. For each of the 10 epochs, training loss and test loss were recorded.

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

As requested for this project, the goal was to use this finalized U-Net model to create predictions for each of the images in the two regions in Guam. Provided that the training and testing losses were reasonable, the finalized U-Net model was used to create predicted images for the unseen images in Guam. A confusion matrix was developed to compare the correctly labeled pixels and incorrectly labeled pixels across the seven habitats. Additionally, side-by-side images were developed of the input satellite image, the ground truth image, and the predicted image using the U-Net model. Finally, the precision, recall, and F1-scores were calculated for each habitat in the class distribution. Accuracy was not considered due to the aforementioned fact that high accuracy could be achieved from a model that predicted Land or Ocean a large majority of the time. Therefore, it is believed that precision and recall are more important than accuracy for this problem.

VGG-16

The second convolutional neural network model considered was the VGG-16 model that was partially provided in the sample code for this project. Figure 2 below shows the model

architecture demonstrated in class. This convolutional neural network model consistently uses 3x3 convolution filters to simplify the architecture and maintain structural uniformity. This process reduces the number of parameters: by using multiple stacked 3x3 filters, this model type increases non-linearity and expressiveness while keeping the parameter count manageable.

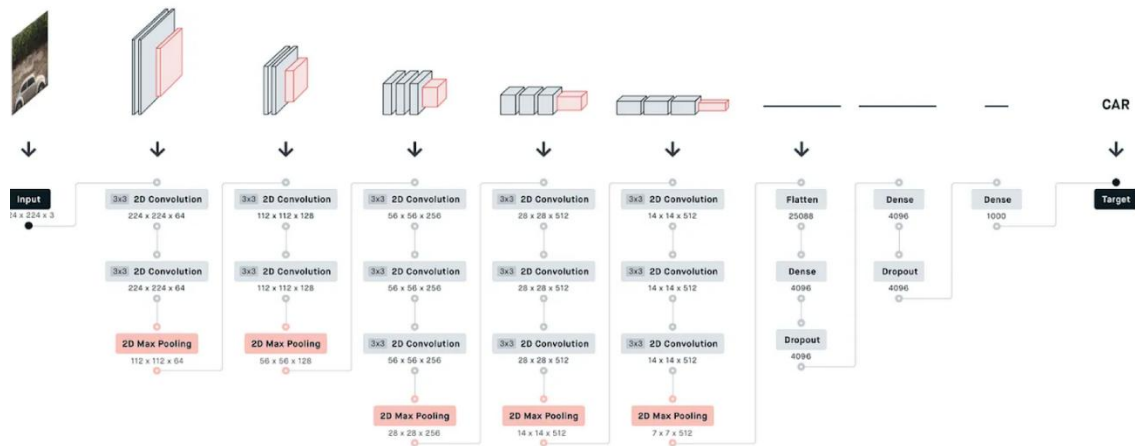


Figure 2: VGG-16 model architecture from lecture, inspiration for the modified VGG-16 model used for image segmentation.

The VGG-16 model code that was created is very similar to the demo code from class. Small adjustments had to be made to match this specific image segmentation task, and some inspiration was drawn from the U-Net model that was already developed. First, there are four encoder stages that extract features from the image. Next, the bottleneck acts as the bridge between the encoder and the decoder, similar to that of the U-Net model above. Finally, the decoder doubles the spatial resolution of the image to rebuild the original pixel-level resolution.

To allow for a comparison between the U-Net and VGG-16 models, many of the training process characteristics for the VGG-16 model were kept the same or made very similar to the characteristics of the U-Net training process. Obviously, the same seven to nine train/test split was used for the images. 10 epochs, 16 batches, 256 pixels for the patch size, and 5 patches per image were used. The same multi-class cross entropy loss function was also used, which allowed for a direct comparison to the trained U-Net model.

Like that of the trained U-Net model, the goal was to use this finalized VGG-16 model to create predictions for each of the images in the two regions in Guam. Provided that the training and testing losses were reasonable, the plan was to use the finalized VGG-16 model to create prediction images for the unseen images in Guam. Additionally, the side-by-side comparison images, confusion matrix, and precision, recall, and F1-scores were also going to be developed for the trained VGG-16 model.

Results and Discussion

U-Net

Table 3 below shows the results from training the U-Net model within this image segmentation task. These results can also be found in the ‘UNETResults’ folder within this project environment. Data includes the number of the epoch, the training loss, and the testing loss. As one can see, the model struggled mightily at first, with both training and testing loss starting out close to 2. As the model shifted to later epochs, the training and testing loss both decreased to under 1. For the 10th and final epoch, the two loss values are very close to each other, suggesting that the model is not overfitting the data. This 10th and final model was saved and used for the prediction step.

Epoch	Training Loss	Testing Loss
1	1.6578	1.9538
2	1.3736	1.7822
3	1.2186	1.6136
4	1.1257	1.5378
5	1.0952	1.1188
6	1.0104	1.0052
7	0.9399	0.8718
8	1.043	0.8617
9	0.9298	1.2956
10	0.8724	0.89

Table 3: The results of training the U-Net model for this image segmentation task.

Figure 3 below shows comparisons between the ground truth images and predicted images from the trained U-Net model, with the original satellite images there for reference. The predicted images on their own can be found as .png files in the ‘UNETPredictionImages’ folder within this project environment. These comparison images below can be found in the ‘UNETPredictionResults’ folder within the project environment.

From the comparisons below, it is clear that the trained U-Net model performed well when it came to predicting the unseen images from Guam. However, the model was not perfect. In the first prediction image, the model had a tough time distinguishing between the Seagrass, Macroalgae, and Sand habitats. In the second prediction image, the model incorrectly predicted the Sand habitat in the places of the Coral and Seagrass habitats. Finally, the third prediction image shows that the trained U-Net model incorrectly predicted the Ocean habitat instead of the Land habitat and the Macroalgae habitat in the places of the Seagrass and Coral habitats. However, the model generally predicted these images very well.

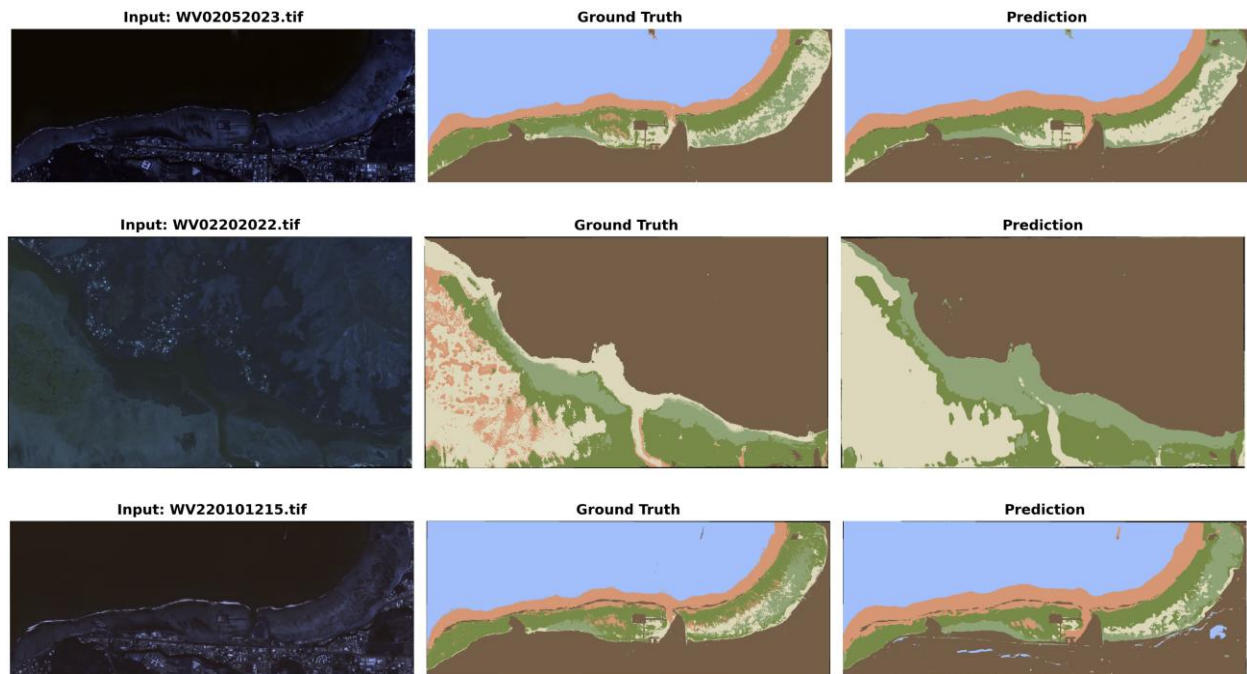


Figure 3: Satellite images, ground truth images, and the prediction images from the trained U-Net model.

Overall, the model achieved a pixel accuracy of 84.62%, which is less important than one may think due to the existing class imbalance. Table 4 below shows the precision, recall, and F1-score for each of the seven classes in the dataset. A full confusion matrix can be found in the ‘UNETPredictionsResults’ folder within this project environment. As seen below, the trained U-Net model had very high performance for the Land, Ocean, and Background classes when it came to making the prediction images. As expected, the biggest struggle was with the Seagrass, Coral, and Sand classes. These habitats, the three smallest by frequency when the Background is excluded, had low precision, recall, and F1-scores compared to the rest of the classes in the distribution. Generally, the model tended to overpredict the Seagrass class and confuse the Coral habitat with the Macroalgae and Sand classes. This suggests that the severe class weight imbalance of the class distribution and the close spectral similarity of the smaller classes were not perfectly addressed.

Number	Name	Color	Recall	Precision	F1-Score
0	Background		0.9659	0.9563	0.9611
1	Seagrass		0.7719	0.3092	0.4415
2	Coral		0.6893	0.5079	0.5849
3	Macroalgae		0.6711	0.7894	0.7255
4	Sand		0.6018	0.6879	0.642
5	Land		0.9653	0.9941	0.9795
6	Ocean		0.8712	0.9941	0.9286

Table 4: Precision, recall, and F1-score for each of the seven habitats when considering the U-Net predicted images and the ground truth images of Guam.

VGG-16

Table 5 below shows the results from training the VGG-16 model within this image segmentation class. These results can also be found in the ‘VGGResults’ folder within this project environment. Data includes the number of the epoch, the training loss, the testing loss, and the run time. Additionally, Figure 4 shows the loss curve across the ten epochs of the training process. Once again, the model struggled heavily at first. However, unlike the U-Net model, the training and testing losses did not decrease significantly with later epochs. The final training and testing losses ended up at 1.2456 and 1.6716 respectively, which were obviously a lot higher than that of the aforementioned U-Net model. A second key issue with this training process was the runtime that lasted for each epoch. The entire training process took quite a while, with many of the epochs reaching close to two minutes in runtime. Considering these poor loss values, and the high run times for each epoch, this finalized VGG-16 model was not moved forward into the prediction step.

Epoch	Training Loss	Testing Loss	Run Time (s)
1	1.8919	1.8230	106.6
2	1.7077	1.6051	107.5
3	1.6264	1.7250	111.3
4	1.5078	1.5745	116.9
5	1.4370	1.394	111.9
6	1.3622	1.573	114.9
7	1.3051	1.5406	108.3
8	1.2518	1.6104	108
9	1.3432	1.5807	107.5
10	1.2456	1.6716	106.5

Table 5: The results of training the VGG-16 model for this image segmentation task.

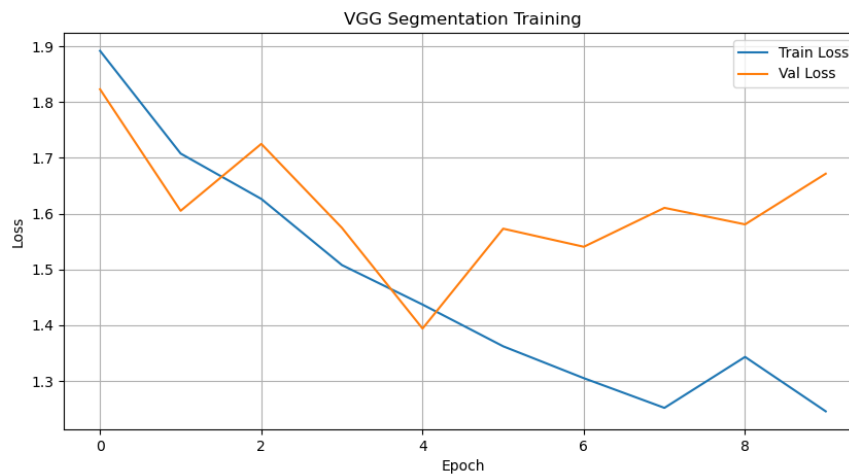


Figure 4: The loss curve in the VGG-16 model segmentation training process.

Conclusion and Future Work

Two different models were created and trained for this image segmentation task. First, a U-Net model was developed using inspiration from an online YouTube video. Second, the VGG-16 model from the demo code was reimagined to match this specific image segmentation task. Both models had high training and testing loss values at the first few epochs of training. However, they had different performances as they moved to the 10th epoch. The U-Net model achieved training and testing losses under 1, while the VGG-16 model maintained training and testing losses well above 1 across all 10 epochs. Therefore, the U-Net model was moved into the prediction step on its own.

When it came to the predictions, the trained U-Net model found decent success. The overall model accuracy for pixels was 84.62%, although that is of less importance than one may think due to the extreme class imbalance. More importantly, the model performed exceptionally well at identifying the Background, Land, and Ocean classes. While the predictions for the Sand, Macroalgae, Coral, and Seagrass habitats were not perfect, the prediction images were still very close to the ground truth images. Modifications are needed in order to solve the class imbalance problem and the issue with the smaller classes having high spectral similarity.

Future work should be done with modified versions of these two models (the U-Net and the VGG-16) or different convolutional neural networks that can be used for image segmentation. For example, the ResNet model was originally considered for this assignment, as it was also recommended in class by the professor. Priority was given to the VGG-16 and U-Net models, as they were considered the most appropriate for this project, but the ResNet model could absolutely be applied to this image segmentation task.

Acknowledgements

I have multiple individuals to thank for helping me with this project. First, I would like to thank Corey Mitterer, a fellow student in CISC484: Introduction to Machine Learning. Corey and I collaborated a lot on this assignment, particularly focusing on the methodology of the three main problems (variable images, class imbalance, and eight spectral bands) and the U-Net model that I created. Corey found the wonderful video from Aladdin Persson that demonstrated to me how to perform PyTorch image segmentation with the U-Net model architecture.

As mentioned above, I also have to thank Aladdin Persson on YouTube for providing a demonstration on how to perform image segmentation in PyTorch using a U-Net model architecture. Although I did have to make several changes to his model to fit this specific task, his YouTube video created a very strong framework to build from. The [video](#), while almost 52 minutes long, is worth the watch to learn more about the U-Net model in PyTorch.

Next, I must address the use of large language models for development for some of the code within this image segmentation project. This project and course allowed for the use of AI in the coding projects, and large language models were definitely necessary for the development of this project. In particular, large language models aided me with training the models and making the predictions using the finalized models. Files that contain some LLM-written code are acknowledged within this project environment through the use of comments.

Finally, I would like to thank Mateo Torres, a former student of CISC484: Introduction to Machine Learning. When he first introduced the project, the professor acknowledged that Mateo did a wonderful job with this project in the previous semester. He recommended Mateo as a helpful resource if anybody needed any extra help. I reached out to Mateo, and he was very gracious enough to offer his help. In particular, Mateo assisted me with ideas for solving the two biggest problems in the project: addressing the variable size images and eight different spectral bands in each image.